Introduccion a Git

Introducción a la Ciencia de Datos · Clase 01

Universidad de Montevideo — FCEE

¿Por qué versionar?

El problema que todos conocen

```
TrabajoFinal.xlsx
TrabajoFinal_v2.xlsx
TrabajoFinal_v2_FINAL.xlsx
TrabajoFinal_v2_FINAL_corregido.xlsx
TrabajoFinal_v2_FINAL_corregido_AHORA_SI.xlsx
copia de TrabajoFinal_v2_FINAL (1).xlsx
```

Seis archivos, ninguna respuesta a las preguntas que importan:

- ¿cuál es la versión buena?
- ¿qué cambió entre una y otra?
- ¿quién lo cambió, y por qué?

Qué es Git

Un programa que lleva **el historial completo de una carpeta**:

- cada versión guardada queda registrada **para siempre**
- cada versión dice **quién**, **cuándo** y **por qué**
- se puede **volver** a cualquier versión anterior
- varias personas trabajan **a la vez** sin pisarse

En este curso Git es **el método de trabajo**.

Los conceptos

El ciclo básico: tres lugares

```
carpeta de trabajo  →  área de preparación  →  historial
  (tus archivos)         (git add)              (git commit)
```

1. **Editás** archivos en tu carpeta, como siempre
2. **Elegís** qué cambios entran en la próxima "foto": `git add`
3. **Sacás la foto** con un mensaje que explica el porqué: `git commit`

   La foto se llama **confirmación** (*commit*). Es la unidad de trabajo del curso: chica, frecuente, con mensaje que se entiende.

Anatomía de una confirmación

```
git commit -m "obtención: agrega descarga del IPC desde el INE"
```

Parte	Qué registra
Autor	quién hizo el trabajo (nombre y mail de **tu** cuenta)
Fecha	cuándo, con hora
Mensaje	**por qué** se hizo el cambio
Contenido	la diferencia exacta, línea por línea

El mensaje importa. “cambios” no dice nada; “limpieza: elimina duplicados de negocio en ventas” sí.

Leer un historial

```
git log --oneline
```

```
a3f21c9 informe: agrega intervalo de confianza al pronóstico
8e07d44 limpieza: trata los precios en cero como faltantes
51b9a02 obtención: agrega descarga del IPC desde el INE
c112f30 estructura inicial del proyecto
```

Se lee de abajo hacia arriba: **es la historia del análisis**. Un tercero puede reconstruir qué se decidió y en qué orden.

Ramas: trabajar sin romper

Una **rama** (*branch*) es una línea de trabajo paralela.

```
git switch -c limpieza-duplicados   # crear rama y pasarse a ella
# ... trabajar y confirmar acá, sin tocar lo estable ...
git switch main                     # volver a la rama principal
git merge limpieza-duplicados       # incorporar el trabajo
```

- `main` siempre queda **funcionando**
- lo experimental vive en ramas hasta que está listo
- en el equipo: **una rama por persona o por tarea**

Conflictos: qué son y por qué no asustan

Dos personas tocaron **las mismas líneas** del mismo archivo. Git no adivina: te muestra las dos versiones y te pide decidir.

```
<<<<<<< HEAD
umbral = 90   # definición de cliente activo: 90 días
=======
umbral = 60   # definición de cliente activo: 60 días
>>>>>>> rama-de-ana
```

Un conflicto no es un error: es una **decisión de método** que el equipo tenía pendiente. Se conversa, se elige, se confirma. (Y se anota en `decisiones.md`.)

Remoto: GitHub

Hasta acá, todo pasó en tu máquina. El **remoto** es la copia compartida en GitHub.

```
git push    # subir mis confirmaciones
git pull    # traer las de los demás
```

- el equipo se sincroniza contra el remoto
- el docente ve el trabajo **cuando ocurre**, no en la entrega
- si tu máquina muere, el trabajo no

Solicitud de incorporación (*pull request*)

Proponer cambios **sin fusionarlos** todavía: otro los revisa primero.

```
rama cap/git-ramas-perez  →  solicitud  →  dos revisiones
       →  correcciones  →  fusión por el docente
```

Es la forma normal de trabajar en cualquier equipo profesional de datos o software.

No todo es terminal

Git es el motor; el volante lo elegís vos

Todas estas herramientas hacen **exactamente el mismo Git**: crean las mismas confirmaciones, el mismo historial, los mismos remotos.

Herramienta	Para quién
Terminal	quien quiere ver qué pasa por dentro
VS Code (panel de control de código fuente)	quien ya edita ahí — la opción más cómoda del curso
RStudio (pestaña Git)	quien trabaja en R
GitHub Desktop	quien prefiere una aplicación dedicada, simple y visual
GitHub web	revisar solicitudes y leer historial desde el navegador

Elegí una, la que te resulte natural. El historial que producen es **idéntico**, y en la evaluación no se distingue cuál usaste.

VS Code: el ciclo completo sin escribir comandos

1. Editás tus archivos, como siempre
2. Panel **Control de código fuente** (icono de ramas, `Ctrl+Shift+G`)
3. **+** junto al archivo = `git add`
4. Escribís el mensaje y **✓ Confirmar** = `git commit`
5. **Sincronizar** = `git push` + `git pull`

   [captura: panel de control de código fuente con un cambio preparado y el mensaje escrito — ver `imagenes/README.md`]

GitHub Desktop: la vista más visual

1. Muestra **la diferencia** de cada archivo, línea por línea
2. Tildás los archivos que entran en la confirmación
3. Mensaje + botón **Commit**
4. **Push origin** para publicar
5. Cambiar de rama es un menú desplegable

   [captura: diferencia de un archivo y el formulario de commit — ver `imagenes/README.md`]

RStudio: la pestaña Git

1. Pestaña **Git** (arriba a la derecha, junto a Environment)
2. Casilla **Staged** = `git add`
3. Botón **Commit** abre la ventana con la diferencia y el mensaje
4. **Push** / **Pull** con las flechas verdes

[captura: pestaña Git con archivos preparados — ver `imagenes/README.md`]

Entonces, ¿para qué aprender los comandos?

- La interfaz alcanza para el **90 %** de los días
- Los comandos aparecen cuando algo se complica: un conflicto enredado, un historial que hay que entender, una máquina ajena
- Y son el **vocabulario común**: la documentación, los foros y los asistentes de IA hablan en comandos.

Las reglas del curso

Qué se versiona y qué no

Sí	No
Código (`.py`, `.R`, `.qmd`)	Datos crudos (`datos/crudo/`)
El script que **obtiene** los datos	Salidas: figuras y tablas (`salidas/`)
`MANIFIESTO.md`, `decisiones.md`	Datos personales — **nunca**
`requirements.txt` / `renv.lock`	Claves y contraseñas — **nunca**

El archivo **`.gitignore`** le dice a Git qué ignorar. Regla del curso: *lo que se puede regenerar, no se versiona.*

La política de autoría

Cada estudiante confirma su propio trabajo, desde su propia cuenta.

- el historial es la evidencia individual de participación
- el integrador del equipo **fusiona**, no es autor de todo

```
git commit --author="Ana Pérez <aperez@um.edu.uy>" -m "..."
```

Un historial donde una sola persona firma todo ⇒ participación mínima **para todo el equipo**.

Los repositorios de cada estudiante

Repositorio	Patrón de Git
`lab-L<n>-<tema>` (equipo)	ramas y fusión
`bitacora-<apellido>` (personal)	repositorio propio, commit semanal

Manos al teclado

Hoy, antes de irse: la primera confirmación

```
# 1. presentarse ante Git (una sola vez por máquina)
git config --global user.name  "Nombre Apellido"
git config --global user.email "usuario@um.edu.uy"

# 2. clonar el repositorio de su equipo
git clone https://github.com/um-icd-2026/lab-L1-macro.git
cd lab-L1-macro

# 3. crear su rama, trabajar, confirmar
git switch -c inicio-apellido
echo "# Notas de <nombre>" > notas_apellido.md
git add notas_apellido.md
git commit -m "estructura: agrega archivo de notas de <nombre>"

# 4. publicar
git push -u origin inicio-apellido
```

O por interfaz, mismo resultado: clonar desde VS Code o GitHub Desktop (paso 2), crear el archivo, preparar → mensaje → confirmar → sincronizar (pasos 3 y 4). El paso 1 sí conviene hacerlo una vez en la terminal.

Los diez comandos del semestre

Comando	Para qué
`git status`	¿en qué estado está todo? (el más usado)
`git add <archivo>`	preparar un cambio
`git commit -m "..."`	confirmar con mensaje
`git log --oneline`	leer la historia
`git diff`	¿qué cambié exactamente?
`git switch -c <rama>`	crear una rama y pasarse
`git merge <rama>`	incorporar una rama
`git push` / `git pull`	sincronizar con GitHub
`git clone <url>`	copiar un repositorio
`git tag hito-1`	etiquetar una entrega

Para profundizar

- Chacon, S. y Straub, B. (2014). *Pro Git* (2.ª ed.). Apress. Gratuito en línea: **https://git-scm.com/book/es** — **en español**.
- La documentación oficial: **https://git-scm.com/docs**
- Capítulos del *book* de la clase: *Git: ramas*, *fusión y conflictos* y *Solicitudes de incorporación y revisión de código*.



Introducción a la Ciencia de Datos · UM

Speaker notes